Score Matching techniques on Goodness of-Fit tests

An internship report for partial fulfillment of the requirements for the Degree of

Master of Technology

in

Communication and Signal Processing

by

Dibyanshu Shekhar Dey

Under the guidance of

Prof. Subhadip Mukherjee

Department of Electronics and Electrical Communication Engineering

INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

Kharagpur - 721302

September 2026

Contents

Abstract

The problem of checking if a sample comes from a known distribution is already well established. For images, real life application involves CT scans, X-rays and more recently generative images. It becomes important to spot the out of distribution (OOD) images especially if the irregular information in images might be too minute, which makes the detection of OOD images quite difficult.

This work tries to go in the direction of addressing these problem. First we use a test statistic, called Kernelized Stein Discrepancy (KSD) that utilizes the score functions to address the hypothesis test. However real life datasets do not involve well defined distributions, which makes the use of score function seemingly impossible. We then utilize another metric - Score Matching, which makes use of the U-Net architecture for image datasets to learn the score function and then we plug-in this learned function into our KSD framework, and do the test all over again.

We then use diffusion posterior sampling which uses the learned score to generate a new set of samples from the test dataset. We then employ the KSD statistic on these generated image dataset to detect the statistical power, and also check quality of these images using PSNR and SSIM curves.

The code of this report can be found on this link `https://github.com/dibyanshushekhardey/Summer_Research_Intern_2026`.

Chapter 1

Introduction

1.1 Background

Kernelized Stein discrepancy is a powerful test statistic that only requires the samples from the unknown dataset and a score function that is based on in-distribution (InD), to maximize the discriminative power between InD and out-of-distribution (OOD). But for images, we require a trained score network to learn the score of the InD samples. Score matching addresses this problem [6]. We then combine these two separate metrics into our work.

1.2 Motivation

Goodness-of-Fit test based literature already many such methods - KSD, MMD, Kolmogorov-Smirnov, Chi-Square, Cramer-von-Mises etc. However most of these tests in statistical field are applied for low dimensional datasets. It becomes important to check their application on multidimensional datasets. KSD tests based on U-statistics already perform substantially well that other tests on toy experiments (eg. 1D Gaussian) [3]. Our motivation simply comes to apply this on multidimensional test setting - images, and analyze them. This leads to score matching which give a method to be combined for multidimensional setting in KSD-U framework.

This work goes like this: first we use the KSD-U Statistic on toy examples, then train a score model based on image dataset (MNIST) and then design an experiment that utilizes both these techniques.

Generative samples can also be tested in our work [1]. We utilize the diffusion posterior sampling to generate new samples using the motivation from the score matching and then use KSD-U framework on them.

1.3 Objectives

1. Develop the KSD-U framework on toy examples.
2. Perform various perturbation based tests on toy examples.
3. Train a score function on the MNIST image dataset.
4. Perform KSD-U on the MNIST dataset.
5. Generate new samples from MNIST dataset and check the quality of such images.
6. Take the best quality images and perform the power of the test on the generated images.

Chapter 2

Kernelized Stein Discrepancy

2.1 Preliminary

A goodness-of-fit test is posed like this - we have two distributions $p(x)$ and $q(x)$, are they same distribution or are they different. Realistic scenarios involve having samples from these two unknown distributions. Two samples tests like MMD involve checking whether two samples come from same distribution or not. However KSD is one sample test. We have samples $x_i \sim p(x)$ where $p(x)$ is unknown. We have score function $s_q(x) = \nabla_x \log q_(x)$ of another distribution $q(x)$. This will be either analytical if $q(x)$ is tractable and we will put the samples from $p(x)$ on this score function or trained on samples from $q(x)$ and then use $p(x)$ samples on the trained score.

2.2 Definitions

The Kernelized Stein discrepancy (KSD) $\mathbb{S}(p, q)$ between distribution p and q is given as

$$\mathbb{S}(p, q) = \mathbb{E}_{x,x' \sim p} \left[\delta_{q,p}(x)^\top k(x, x') \delta_{q,p}(x') \right] \tag{2.1}$$

where $\delta_{q,p}(x) = s_q(x) - s_p(x)$ is definedas the score difference between p and q, and x, x' are i.i.d. draws from $p(x)$ [3].

However we use a KSD estimate if we have the samples from $p(x)$. We use Theorem 3.6 [3] to define the function

$$\begin{aligned} u_q(x, x') = s_q(x)^\top k(x, x') s_q(x') + s_q(x)^\top \nabla_{x'} k(x, x') \\ + \nabla_x k(x, x')^\top s_q(x') + \operatorname{trace} \left(\nabla_{x,x'} k(x, x') \right) \end{aligned} \tag{2.2}$$

The kernel $k(x, x')$ can be any suitable positive definite kernel. Here, we use the RBF kernel:

$$k(x, x') = \exp \left(-\frac{\|x - x'\|^2}{2h^2} \right), \tag{2.3}$$

where h is the bandwidth parameter. In particular, we use

$$h = \sqrt{\frac{D}{2}}. \tag{2.4}$$

The bandwidth can also be selected using a median heuristic [2]

$$D = \operatorname{median} \left\{ \|x_i - x_j\| \right\}_{i \neq j}. \tag{2.5}$$

2.3 KSD-U Statistic

Now the KSD-U uses the U-statistic approach to estimate KSD which is given as

$$\widehat{\mathbb{S}}_u(p,q) = \frac{1}{n(n-1)} \sum_{1 \le i \ne j \le n} u_q(x_i, x_j). \tag{2.6}$$

However, to perform the hypothesis test, we use the bootstrap estimation

$$\widehat{\mathbb{S}}_u^*(p,q) = \sum_{i \ne j} \left(w_i - \frac{1}{n}\right)\left(w_j - \frac{1}{n}\right) u_q(x_i, x_j), \tag{2.7}$$

with

$$(w_1, \ldots, w_n) \sim \text{Mult}\left(n; \frac{1}{n}, \ldots, \frac{1}{n}\right), \tag{2.8}$$

where $w_1, \ldots, w_n$ are multinomial weights.

2.4 KSD Goodness-of-Fit Algorithm

Algorithm 1 KSD-U bootstrap Goodness-of-fit test[3]

Input: Sample $\{x_i\}$ and score function $s_q(x) = \nabla_x \log q(x)$. Bootstrap sample size m.

Test: H_0: $\{x_i\}$ is drawn from q vs. H_1: $\{x_i\}$ is drawn from p.

1: Compute $\widehat{\mathbb{S}}_u$ by and $u_q(x, x')$ as defined in Eq. (2.2). Generate m bootstrap samples $\widehat{\mathbb{S}}_u^*$ by Eq. (2.7).

2: Reject H_0 with significance level α if the percentage of $\widehat{\mathbb{S}}_u^*$ that satisfies

$$\widehat{\mathbb{S}}_u^* > \widehat{\mathbb{S}}_u$$

is less than α.

2.5 Experiments

Consider the hypotheses $H_0 : X \sim \mathcal{N}(0, \sigma^2)$ vs $H_1 : X \sim \mathcal{N}(1, \sigma^2)$. We calculate the AUC score. For the single-mode setup, we consider

1. different means that vary according to $\delta \in \{0,\ 0.2,\ 0.4,\ 0.6,\ 0.8,\ 1.0\}$ keeping σ same as 1.0 for alternate.

2. different variances $\sigma \in \{0.5,\ 0.75,\ 1.0,\ 1.25,\ 1.5, 2.0\}$ keeping the alternate mean fixed as same as null.

.

We repeat the different means and variances for a different numbers of trials. The AUC is then calculated for the corresponding hypothesis tests with their corresponding ROC curves plotted along with the error rates.

We find that the KSD test is able to distinguish between the two distributions very well even when we vary the means and the sigmas, except when it is null. The error rates also decreases significantly if we take sufficient number of samples and perform the tests.

Table 2.1: AUC Curves for Single Mode Gaussian Distributions

Deltas δ (fixed variance = 1)	AUC	Sigmas σ (fixed mean = 0)	AUC
0	0.516	0.5	0.995
0.2	0.983	0.75	0.995
0.4	1.0	1.0	0.4828
0.6	0.985	1.25	0.995
0.8	0.975	1.5	0.990
1.0	1.0	1.75	0.985

Table 2.2: Error rate for different sample sizes and values of δ.

Number of samples	$\boldsymbol{\delta = 0.25}$	$\boldsymbol{\delta = 0.5}$	$\boldsymbol{\delta = 0.75}$	$\boldsymbol{\delta = 1.0}$	$\boldsymbol{\delta = 1.5}$
25	0.475	0.370	0.200	0.060	0.030
50	0.485	0.245	0.050	0.035	0.030
75	0.400	0.110	0.025	0.035	0.030
100	0.360	0.090	0.040	0.020	0.025
150	0.300	0.030	0.040	0.040	0.020
200	0.205	0.025	0.025	0.045	0.035
300	0.130	0.025	0.020	0.020	0.020
500	0.035	0.025	0.025	0.040	0.025
750	0.020	0.035	0.015	0.025	0.020
1000	0.020	0.035	0.040	0.020	0.030

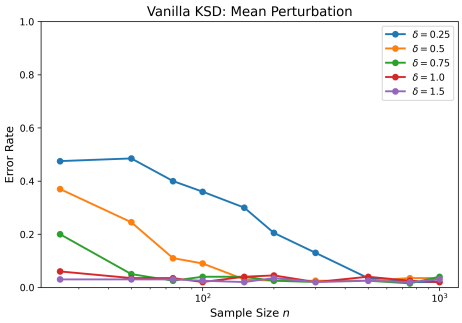
Number of samples	$\boldsymbol{\sigma = 0.5}$	$\boldsymbol{\sigma = 0.75}$	$\boldsymbol{\sigma = 1.0}$	$\boldsymbol{\sigma = 1.25}$	$\boldsymbol{\sigma = 1.5}$	$\boldsymbol{\sigma = 2.0}$
25	0.405	0.475	0.510	0.450	0.310	0.145
50	0.095	0.460	0.505	0.400	0.185	0.030
75	0.025	0.420	0.505	0.375	0.055	0.030
100	0.010	0.345	0.500	0.305	0.050	0.015
150	0.020	0.195	0.475	0.215	0.030	0.030
200	0.040	0.115	0.540	0.115	0.015	0.020
300	0.020	0.055	0.490	0.055	0.035	0.035
500	0.025	0.035	0.485	0.030	0.025	0.020
750	0.035	0.015	0.490	0.005	0.040	0.020
1000	0.035	0.025	0.510	0.030	0.015	0.035

Table 2.3: Error rate for different sample sizes and values of σ.

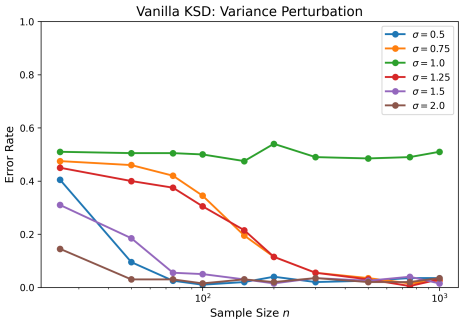
2.6 Discussion

We went through the toy example setting and used different perturbation techniques on single mode Gaussian mixture model. However, even for this case, the number of samples being taken is very high for the error rate to reduce. Also, this is quite a simple experiment, having to distinguish between two single mode distributions. The future experiments have involve multi modal GMM in which the standard KSD fails [4]. We then have used the motivation from this paper and have used different noise scales to perturb the samples and perform the **Pooled KSD** . This is an ongoing experiment.

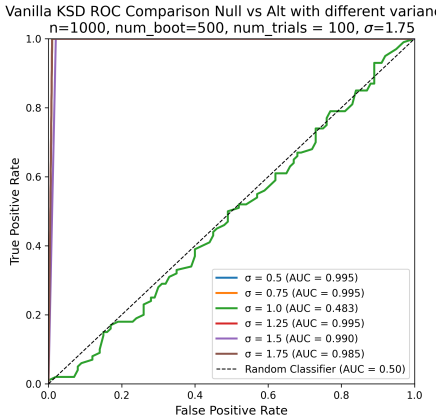
We also have used standard RBF Kernel, which is a fixed kernel, and does not improve upon the statistical power of the test for multidimensional setting. We are currently working upon this so that this kernel also becomes a learned kernel. We have not performed the tests using standard IMQ kernel.



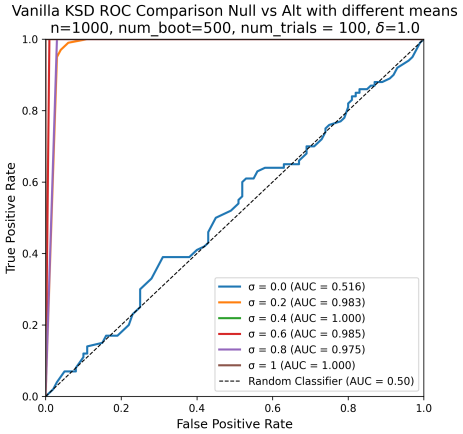
(a)



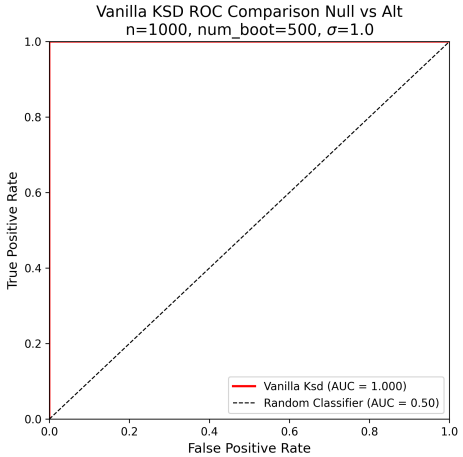
(b)



(c)



(d)



(e)

Figure 2.1: Comparison of the proposed method under different experimental settings.

Chapter 3

Score Matching technique on KSD Framework

3.1 Score Matching Objective

For simple distributions which are tractable, we can easily find the score function and use it analytically. It acts as a sanity check for any algorithm. However for distributions that are intractable, finding the score becomes difficult. We use score matching techniques that utilizes a neural network to train the score function to be used later. We use, in our work the score matching techniques using stochastic differential equations [6]. For training purpose, we use forward SDE .

$$\mathrm{d}x = f(x,t)\,\mathrm{d}t + g(t)\,\mathrm{d}w \tag{3.1}$$

.
Following the paper [6] and tutorial based on it, we use

$$dx = \sigma^t dw, \quad t \in [0,1] \tag{3.2}$$

to follow the SDE. We then draw samples from $x(t) \sim q_(x)$ and perturb it with $x_t = x_0 + \sigma_t z$, $z \sim \mathcal{N}(0, I)$. We then choose σ^t as follows

$$\sigma_t = \left(\frac{\sigma^{2t}-1}{2\log\sigma}\right)^{1/2} \tag{3.3}$$

The perturbed distribution becomes

$$q_{0t}(x_t \mid x_0) = \mathcal{N}\left(x_t; x_0, \frac{1}{2\log\sigma}(\sigma^{2t}-1)\mathbf{I}\right) \tag{3.4}$$

We can approximate the score function as

$$s_\theta(x_t,t) \approx \nabla_{x_t} \log q_t(x_t). \tag{3.5}$$

However we only have the access to the perturbed conditioned true samples. Using marginals and expectation, we can approximate the marginals i.e. our perturbed samples as conditional score. The conditional score is

$$\nabla_{x_t} \log q_{0t}(x_t \mid x_0) = -\frac{1}{2\sigma_t^2}\nabla_{x_t}\|x_t - x_0\|^2 \tag{3.6}$$

$$= -\frac{1}{\sigma_t^2}(x_t - x_0) \tag{3.7}$$

$$= -\frac{z}{\sigma_t}. \tag{3.8}$$

We then make use of Eq. (7) [6] for training our score function, given as

$$\theta^* = \arg\min_\theta \mathbb{E}_t\left\{\sigma_t \mathbb{E}_{x_0}\mathbb{E}_{x_t|x_0}\left[\|s_\theta(x_t,t) - \nabla_{x_t}\log q_{0t}(x_t \mid x_t)\|_2^2\right]\right\}. \tag{3.9}$$

We then use 3.8 in 3.9 for our training of the model. After we get this trained score model we plug the trained score in 2.2 which becomes

$$\begin{aligned} u_\theta(x, x'; t) = s_\theta(x, t)^\top k(x, x') s_\theta(x', t) \\ + s_\theta(x, t)^\top \nabla_{x'} k(x, x') \\ + \nabla_x k(x, x')^\top s_\theta(x', t) \\ + \operatorname{tr}\left(\nabla_{x,x'} k(x, x')\right). \end{aligned} \tag{3.10}$$

3.2 Experiment

We now use the training objective on the MNIST dataset. We train the score on the MNIST even training dataset and now we use the score based KSD framework on the following hypothesis: Null - even digits vs Alternate - odd digits. The selection of σ and timestep t are our hyper-parameters. We follow [6] and keep $\sigma = 25$. However to get the best possible results, we take $t \in (0.00001, 1)$ logarithmically. We then focus near the region $t = 0.1$ and then do further experiments to narrow it down to $t \in \{0.25, 0.27, 0.3, 0.33, 0.35, 0.37, 0.4\}$. 200 bootstrap samples along with 128 original samples across 100 trials are taken in account. The AUC Results are given below.

Table 3.1: AUC values for different values of t.

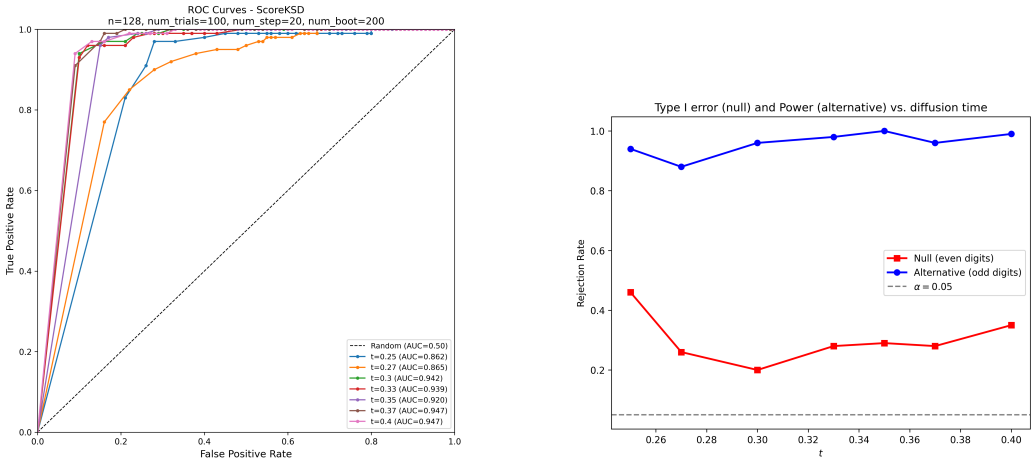
Timestep t	AUC
0.25	0.862
0.27	0.865
0.30	0.942
0.33	0.939
0.35	0.920
0.37	0.947
0.40	0.947

We then perform the power test on null and alternate samples to test the statistical power of the tests. The results are provided in the Fig 3.1

3.3 Discussion

The score based KSD framework works well for MNIST dataset. It is also able to reasonably perform well on the power tests for null and alternate. However the power is still not close to the desired level for both.

There are two problems that are arising: selection of σ and t. For other datasets,



(a) (b)

Figure 3.1: ROC Curves (a) and the Power plots (b) for MNIST Even vs Odd samples

we might have to tune it, but given the increase in dimension it is still now clear how computationally expensive it might become.

Chapter 4

Diffusion Posterior Sampling

4.1 Preliminaries

Now that we have a trained score model, we want to generate samples based on that. We get our inspiration from diffusion posterior sampling, which is used in noisy inverse problems. In such case, we want to get an estimate of the clean images when conditioned on the noisy observation [1]. The paper uses discrete denoising process, this work tries to implement Euler - Maruyama process, discussed in [5].

4.2 Diffusion Posterior Sampling

We first consider an inverse problem of the form

$$y = \mathcal{A}(x_0) + \epsilon, \tag{4.1}$$

where x_0 denotes the unknown clean image, $\mathcal{A}$ is a measurement operator, y is the observed measurement, and ϵ represents measurement noise. Our objective is to generate samples from the posterior distribution

$$q(x_0 \mid y) \propto q(y \mid x_0)p(x_0). \tag{4.2}$$

A score-based model provides an estimate of the score of the perturbed data distribution which we have already done,

$$s_\theta(x_t, t) \approx \nabla_{x_t} \log q_t(x_t), \tag{4.3}$$

where $p_t(x_t)$ denotes the marginal distribution of the data at diffusion time t 3.5.

4.2.1 Forward Perturbation Process and Training

This process has been used in training the score network (See Section 3.1).

4.2.2 Reverse-Time Sampling

Once the score network has been trained, samples can be generated by solving the reverse-time SDE. The reverse-time SDE is

$$dx = \left[f(x,t) - g_t^2 \nabla_x \log p_t(x)\right] dt + g_t d\overline{W}_t, \tag{4.4}$$

where time is integrated from T towards 0 [6].

Replacing the unknown score by the trained score network gives

$$dx = \left[f(x,t) - g(t)^2 s_\theta(x,t)\right] dt + g(t) d\overline{W}_t. \tag{4.5}$$

In the implementation, an Euler-Maruyama discretization is used. With a reverse time step $dt > 0$, the update is

$$x'_{t-\mathrm{d}t} = x_t + g_t^2 s_\theta(x_t, t)\,\mathrm{d}t + g_t\sqrt{\mathrm{d}t}\,z. \tag{4.6}$$

where $z \sim \mathcal{N}(0, I)$.

4.2.3 Tweedie's Estimate

At each diffusion time, an estimate of the corresponding clean image is obtained using Tweedie's formula:

$$\hat{x}_0 = x_t + \sigma_t^2 s_\theta(x_t, t). \tag{4.7}$$

This estimate is subsequently used to evaluate the measurement consistency.

4.2.4 DPS Correction and step size

For the inverse problem, the estimated clean image should satisfy

$$y \approx \mathcal{A}(\hat{x}_0). \tag{4.8}$$

A measurement loss is therefore defined as

$$\mathcal{L}_{\text{meas}} = \|y - \mathcal{A}(\hat{x}_0)\|_2^2. \tag{4.9}$$

Since $\hat{x}_0$ depends on x_t, the gradient of the measurement loss with respect to the current noisy state is given as

$$g_t = \nabla_{x_t} \|y - \mathcal{A}(\hat{x}_0)\|_2^2. \tag{4.10}$$

The DPS correction moves the reverse-SDE sample in the direction that tries to reduce the measurement error:

$$x_{t-dt} = x'_{t-dt} - \zeta_t \nabla_{x_t} \|y - \mathcal{A}(\hat{x}_0)\|_2^2. \tag{4.11}$$

In the paper implementation [1], the adaptive guidance step size is

$$\zeta_t = \frac{\zeta'}{\|y - \mathcal{A}(\hat{x}_0)\|_2 + \epsilon}, \tag{4.12}$$

where ζ' is a guidance parameter which is defined by us and ϵ is a small numerical constant.

4.3 DPS Algorithm

Algorithm 2 DPS-Gaussian VE-SDE

Input: y

Sample $x_1 \sim p_1(x)$, ζ'
for $t : 1 \to 0$ **do**
  $s_t \leftarrow s_\theta(x_t, t)$
  $\hat{x}_0 \leftarrow x_t + \sigma_t^2 s_t$
  $\mathcal{L}_{\text{meas}} \leftarrow \|y - \mathcal{A}(\hat{x}_0)\|^2$
  $g_t \leftarrow \nabla_{x_t} \mathcal{L}_{\text{meas}}$
  $x'_{t-dt} \leftarrow x_t + g_t^2 s_t \, dt + g_t \sqrt{dt} \, z$
  $\zeta_t \leftarrow \dfrac{\zeta'}{\|y - \mathcal{A}(\hat{x}_0)\| + \epsilon}$
  $x_{t-dt} \leftarrow x'_{t-dt} - \zeta_t g_t$
end for

4.4 Experiments

We consider two cases for noising a clean image - Gaussian and Inpainting.

In Gaussian setting, we use $\zeta' \in \{0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1, 5, 10\}$ and generate diffusion posterior samples. We then compare the clean image with measurement and posterior mean for $\zeta' = 0.5$. We then find PSNR and SSIM values for these different ζ'.

In Inpainting setting, we in-paint 20% and we again use the same ζ' and generate diffusion posterior samples. We then compare the clean image with measurement and posterior mean for $\zeta' = 0.1$. We then find PSNR and SSIM values for these different ζ'.

4.4.1 Gaussian

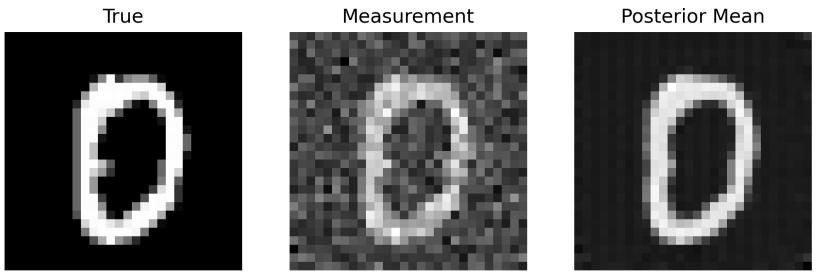


Figure 4.1: Clean, Measurement and Posterior Mean

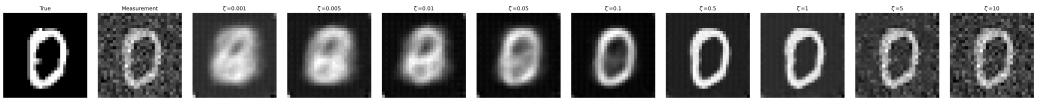
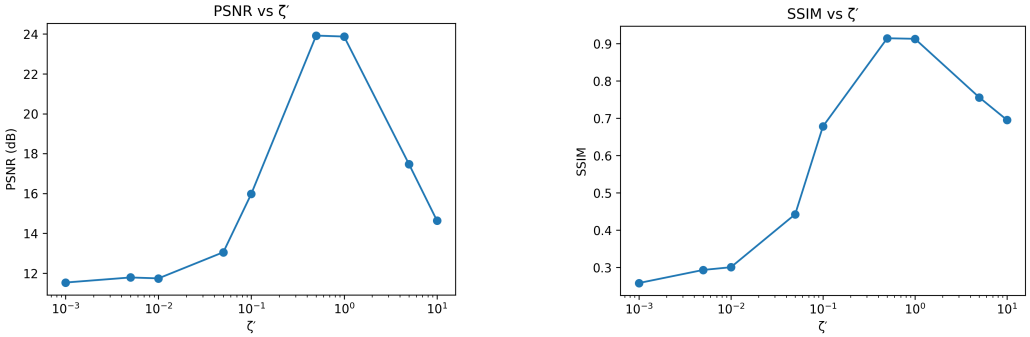


Figure 4.2: Posterior mean images for different ζ'



(a) Gaussian PSNR

(b) Gaussian SSIM

Figure 4.3: PSNR and SSIM for different ζ' for Gaussian

4.4.2 Inpainting

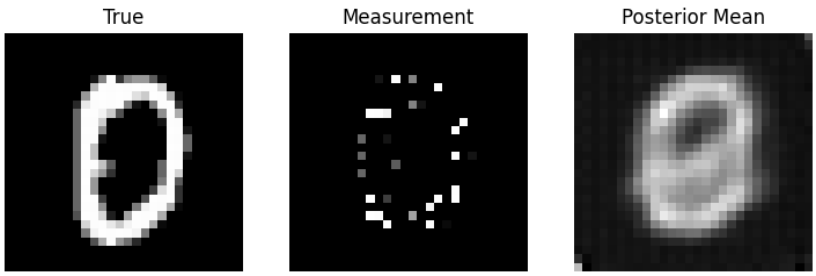


Figure 4.4: Clean, Measurement and Posterior Mean

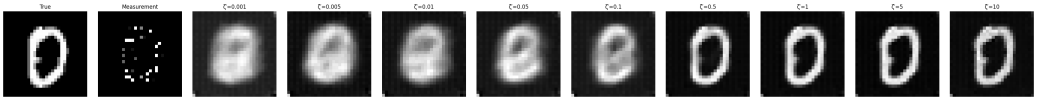
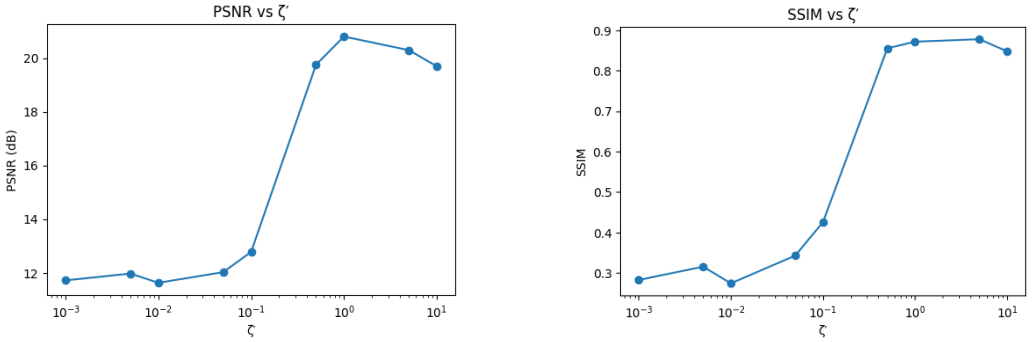


Figure 4.5: Posterior mean images for different ζ'



(a) Inpainting PSNR

(b) Inpainting SSIM

Figure 4.6: PSNR and SSIM for different ζ' for Inpainting

Table 4.1: PSNR and SSIM for different values of ζ' under Gaussian and inpainting settings.

ζ'	**Gaussian**		**Inpainting**	
	PSNR (dB)	**SSIM**	**PSNR (dB)**	**SSIM**
0.001	11.69	0.2826	11.72	0.2828
0.005	11.61	0.2861	11.97	0.3156
0.01	11.77	0.2893	11.63	0.2744
0.05	13.37	0.4660	12.03	0.3429
0.1	15.84	0.6724	12.78	0.4255
0.5	23.83	0.9162	19.75	0.8553
1	23.93	0.9109	20.80	0.8717
5	17.57	0.7583	20.30	0.8778
10	14.66	0.6957	19.70	0.8478

Chapter 5

KSD Framework on Diffusion Posterior Sampling

5.1 Experiments

Based on all the previous chapters, we now use the KSD framework on the diffusion posterior samples. We now generate noisy images for the fist 10 images of the MNIST even data set using the Gaussian noising. We then use the KSD framework to compute the statistical power of the test for true samples and the DPS samples.

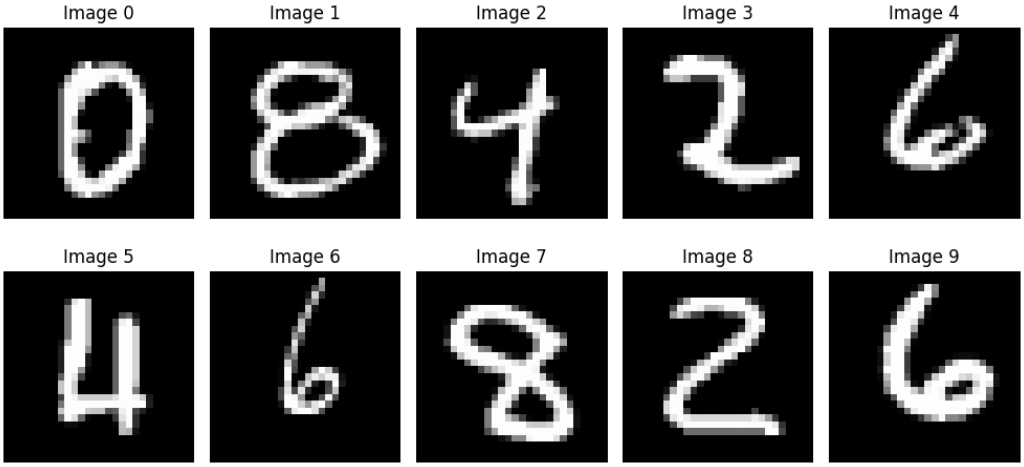


Figure 5.1: Clean 10 images from MNIST

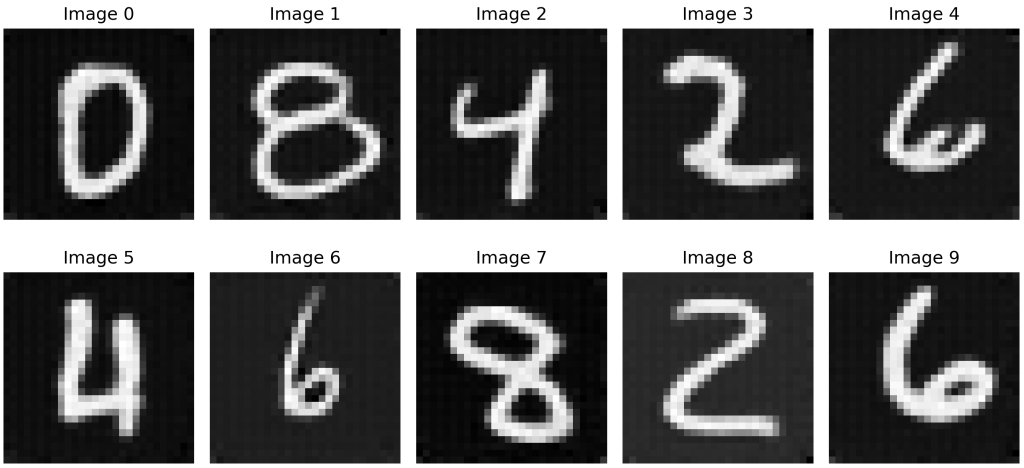


Figure 5.2: Gaussian posterior means 10 images

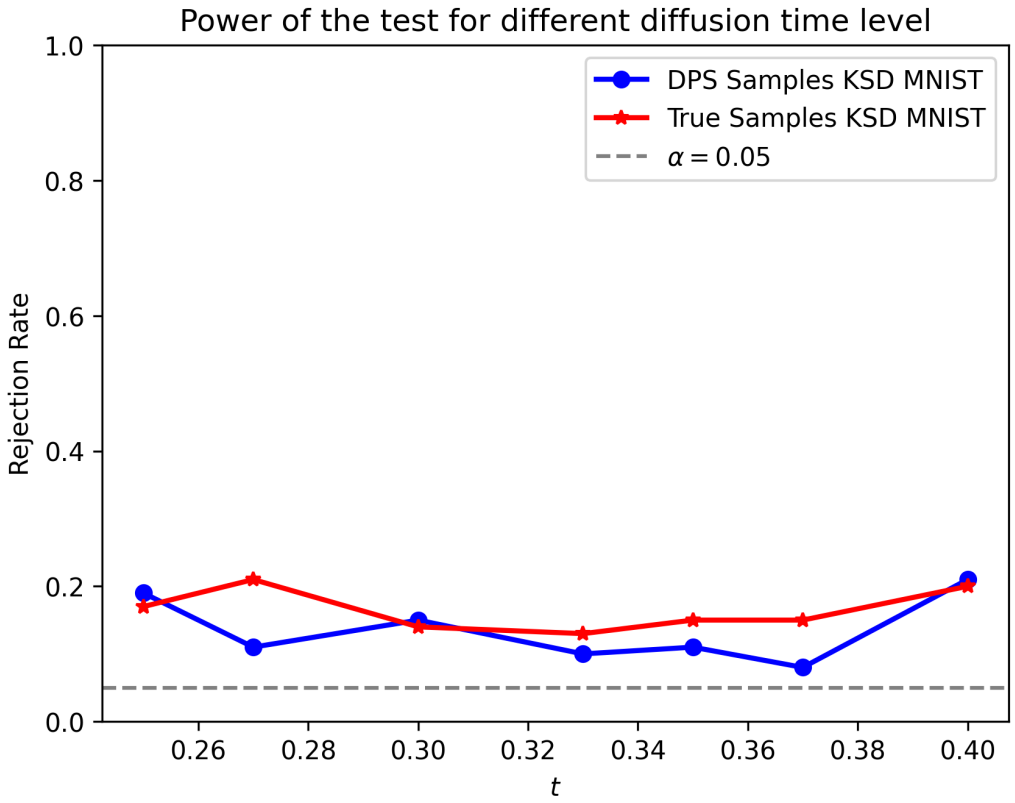


Figure 5.3: KSD Power test for DPS Clean samples and True Samples

5.2 Discussion

We have applied the KSD framework on the DPS samples and we find that the overall power is similar to bith true and DPS samples, however they are still not close to the desired power level.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

We have applied the KSD framework on single mode Gaussian mixture model, and with a trained score network used it on MNIST dataset, which gives us fairly reasonable results. We then used diffusion posterior sampling approach to generate new samples using the same score network and again applied the KSD framework on those DPS samples, which again gave fairly reasonable results.

Based on these, we can say that the KSD framework can be applied on multidimensional setups. However it is still computationally expensive, and the power of the statistical tests can still be made to improve.

6.2 Future Work

We can try to use different tests like MMD to get better statistical power. The use of perturbed kernel shows promise for multi-modal setting, however it needs to be applied on image datasets. The use of learned kernel is also helpful which theoretically allows us to increase the power of the test.

References

[1] Hyungjin Chung et al. *Diffusion Posterior Sampling for General Noisy Inverse Problems*. 2024. arXiv: 2209.14687 [stat.ML]. URL: https://arxiv.org/abs/2209.14687.

[2] Damien Garreau, Wittawat Jitkrittum, and Motonobu Kanagawa. *Large sample analysis of the median heuristic*. 2018. arXiv: 1707.07269 [math.ST]. URL: https://arxiv.org/abs/1707.07269.

[3] Qiang Liu, Jason D. Lee, and Michael I. Jordan. *A Kernelized Stein Discrepancy for Goodness-of-fit Tests and Model Evaluation*. 2016. arXiv: 1602.03253 [stat.ML]. URL: https://arxiv.org/abs/1602.03253.

[4] Xing Liu, Andrew B. Duncan, and Axel Gandy. *Using Perturbation to Improve Goodness-of-Fit Tests based on Kernelized Stein Discrepancy*. 2023. arXiv: 2304.14762 [stat.ML]. URL: https://arxiv.org/abs/2304.14762.

[5] Yang Song. *Score-Based Generative Modeling*. https://yang-song.net/blog/2021/score/. 2021.

[6] Yang Song et al. *Score-Based Generative Modeling through Stochastic Differential Equations*. 2021. arXiv: 2011.13456 [cs.LG]. URL: https://arxiv.org/abs/2011.13456.